

AULA 006

Alunos:

Pedro Lucas Gomes Braido – RA: 24190443-2

Leonardo Fratoni Borges da Silva – RA: 24047610-2

1 – JavaScript

JS index.js × +	Console I/O AI Agent
<pre>1 console.log(0.1 * 3 === 0.3); 2 console.log(9007199254740993);</pre>	<pre>false 9007199254740992</pre>

Detecção: nunca.

2 – Python

main.py × +	Console I/O AI Agent
<pre>1 p = "maçã" 2 print(len(p), len(p.encode()))</pre>	<pre>4 6</pre>

Detecção: nunca.

3 – Go

main.go ×	Console I/O AI Agent
<pre>1 package main 2 import "fmt" 3 4 func main() { 5 var b byte = 255 6 b++ 7 fmt.Println(b) 8 }</pre>	<pre>0</pre>

Detecção: nunca.

4 – Java

```
J Main.java x +
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int[] v = new int[3];
6         System.out.println(v[0]);
7         System.out.println(v[3]);
8     }
9 }
```

```
Console I/O AI Agent Exit 1
0
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds
for length 3
    at Main.main(Main.java:7)
Debug this error
```

Detecção: execução.

5 – Rust

```
HelloWorld.rs x +
1 fn main() {
2     let s = String::from("oi");
3     let t = s;
4     println!("{}", s, t);
5 }
```

```
Console I/O AI Agent Exit 1
error[E0382]: borrow of moved value: `s`
--> HelloWorld.rs:4:23
|
2 |     let s = String::from("oi");
|         - move occurs because `s` has type `String`, which
|         does not implement the `Copy` trait
3 |     let t = s;
|         - value moved here
4 |     println!("{}", s, t);
|               ^ value borrowed here after move
|
help: consider cloning the value if the performance cost is
acceptable
|
3 |     let t = s.clone();
|               ++++++++

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain
E0382`.
Debug this error
```

Detecção: compilação.

6 – C

```
C Main.c x +
1 #include <stdio.h>
2 int main()
3 {
4     union { int i; float f; } u;
5     u.f = 1.0f;
6     printf("%d\n", u.i);
7 }
```

```
Console I/O AI Agent
1065353216
```

Detecção: nunca, em implementações comuns.